

Mongoose Cheat Sheet

A complete reference of Mongoose model methods, query modifiers, and schema features — with simple examples and plain-language explanations.

1. Creating Documents

Method	Example	What it does
<code>.create()</code>	<pre>await User.create({ email, password })</pre>	Builds and saves a brand new document in one step. Runs schema validators and <code>pre("save")</code> hooks.
<code>new Model() + .save()</code>	<pre>const u = new User({ email }); await u.save();</pre>	Builds a document in memory first (lets you modify it before saving), then saves it. Also runs validators and hooks.
<code>.insertMany()</code>	<pre>await User.insertMany([{... }, { ... }])</pre>	Inserts multiple documents in one batch call. Faster than looping <code>.create()</code> one at a time. Skips <code>pre("save")</code> hooks by default — only <code>pre("insertMany")</code> hooks run.

2. Reading / Finding Documents

Method	Example	What it does
<code>.find()</code>	<pre>await User.find({ role: "artist" })</pre>	Returns an array of all documents matching the filter. Empty filter <code>{}</code> returns everything.
<code>.findOne()</code>	<pre>await User.findOne({ email })</pre>	Returns the first single document matching the filter, or <code>null</code> if none found.
<code>.findById()</code>	<pre>await User.findById(userId)</pre>	Shorthand for <code>findOne({ _id: userId })</code> . Only works with the <code>_id</code> field.
<code>.countDocuments()</code>	<pre>await User.countDocuments({ role: "artist" })</pre>	Returns the number of documents matching the filter, not the documents themselves.
<code>.exists()</code>	<pre>await User.exists({ email })</pre>	Returns <code>{ _id: ... }</code> if a match exists, or <code>null</code> . Faster than <code>.findOne()</code> when you only need a yes/no answer.
<code>.distinct()</code>	<pre>await User.distinct("role")</pre>	Returns an array of all unique values for a given field across matching documents.

3. Updating Documents

Method	Example	What it does
<code>.findByIdAndUpdate()</code>	<pre>await User.findByIdAndUpdate(id, { \$set: { name } }, { returnDocument: 'after' })</pre>	Finds by <code>_id</code> and updates it in one atomic step. Skips schema validators unless <code>runValidators: true</code> is set. Does not trigger <code>pre("save")</code> hooks.
<code>.findOneAndUpdate()</code>	<pre>await Fav.findOneAndUpdate({ userId }, { \$addToSet: { list: id } }, { upsert: true })</pre>	Same as above but matches by any filter, not just <code>_id</code> . Supports <code>upsert: true</code> to create the document if no match is found.
<code>.updateOne()</code>	<pre>await User.updateOne({ email }, { \$set: { isOnline: true } })</pre>	Updates the first matching document but does not return it — just a status report (<code>matchedCount</code> , <code>modifiedCount</code>).
<code>.updateMany()</code>	<pre>await User.updateMany({ role: "student" }, { \$set: { isActive: true } })</pre>	Updates every document matching the filter, not just the first one.
<code>document.save()</code>	<pre>user.password = newPass; await user.save();</pre>	Fetch a real document first, modify its fields in memory, then persist. Only method that

runs `pre("save")` hooks and `isModified()` checks.

Common Update Operators (used inside `$set`, etc.)

Operator	Example	What it does
<code>\$set</code>	<code>{ \$set: { title: "New" } }</code>	Sets/replaces the value of a field.
<code>\$push</code>	<code>{ \$push: { list: id } }</code>	Adds a value to an array — always, even if it's a duplicate.
<code>\$addToSet</code>	<code>{ \$addToSet: { list: id } }</code>	Adds a value to an array only if it isn't already present (no duplicates).
<code>\$each</code>	<code>{ \$addToSet: { list: { \$each: [id1, id2] } } }</code>	Used with <code>\$push</code> / <code>\$addToSet</code> to add multiple values at once instead of one.
<code>\$pull</code>	<code>{ \$pull: { list: id } }</code>	Removes all instances of a value from an array.
<code>\$inc</code>	<code>{ \$inc: { views: 1 } }</code>	Increments (or decrements, with a negative number) a numeric field.
<code>\$unset</code>	<code>{ \$unset: { bio: "" } }</code>	Removes a field from the document entirely.

4. Deleting Documents

Method	Example	What it does
<code>.findByIdAndDelete()</code>	<code>await User.findByIdAndDelete(id)</code>	Finds by <code>id</code> , deletes it, and returns the deleted document.
<code>.findOneAndDelete()</code>	<code>await Music.findOneAndDelete({ authorId })</code>	Same but matches by any filter. Returns the deleted document.
<code>.deleteOne()</code>	<code>await User.deleteOne({ email })</code>	Deletes the first match. Returns only a status report, not the document.
<code>.deleteMany()</code>	<code>await Music.deleteMany({ authorId })</code>	Deletes every matching document — e.g. remove all songs by one artist.

5. Query Modifiers (chained onto find methods)

Modifier	Example	What it does
<code>.select()</code>	<code>User.find().select("email role")</code>	Limits which fields are returned. Prefix with <code>-</code> to exclude, e.g. <code>.select("-password")</code> .
<code>.sort()</code>	<code>User.find().sort({ createdAt: -1 })</code>	Orders results. <code>1</code> = ascending, <code>-1</code> = descending.
<code>.limit()</code>	<code>User.find().limit(10)</code>	Caps the number of documents returned.
<code>.skip()</code>	<code>User.find().skip(20).limit(10)</code>	Skips the first N documents — used for pagination.
<code>.populate()</code>	<code>Album.find().populate("musicList")</code>	Replaces <code>ObjectId</code> references with the actual referenced documents.
<code>.lean()</code>	<code>User.find().lean()</code>	Returns plain JavaScript objects instead of full Mongoose documents — faster, but loses document methods like <code>.save()</code> and <code>virtuals</code> .

6. Document Instance Properties & Methods

These only exist on real documents (from `.save()` / `new Model()` / `.findOne()`), not on plain query results with `.lean()`.

Property/Method	Example	What it does
-----------------	---------	--------------

<code>.isModified()</code>	<code>this.isModified("password")</code>	Returns <code>true</code> if a field has been changed since the document was loaded/created. Used inside <code>pre("save")</code> hooks to skip unnecessary work.
<code>.isNew</code>	<code>if (user.isNew) {...}</code>	<code>true</code> if the document hasn't been saved to the database yet.
<code>._id</code>	<code>user._id</code>	The document's unique MongoDB identifier (an <code>ObjectId</code>).
<code>.toObject()</code>	<code>user.toObject()</code>	Converts a Mongoose document into a plain JS object.
<code>.toJSON()</code>	<code>user.toJSON()</code>	Similar to <code>.toObject()</code> , automatically used when a document is passed to <code>res.json()</code> .
<code>.equals()</code>	<code>id1.equals(id2)</code>	Correctly compares two <code>ObjectId</code> s by value (safer than <code>===</code> , which compares by reference).

7. Schema-Level Features

Feature	Example	What it does
<code>pre("save")</code>	<code>schema.pre("save", function(next) {...})</code>	Runs custom logic before a document is saved via <code>.save()</code> / <code>.create()</code> . Common use: password hashing.
<code>post("save")</code>	<code>schema.pre("save", function(doc) {...})</code>	Runs after a document has been saved — e.g. sending a welcome email.
<code>pre("findOneAndUpdate")</code>	<code>schema.pre("findOneAndUpdate", function(next) {...})</code>	Runs before a query-based update. <code>this</code> refers to the query , not the document, so <code>isModified()</code> isn't available here.
methods	<code>schema.methods.comparePassword = function(pw) {...}</code>	Adds a custom method callable on document instances : <code>user.comparePassword(pw)</code> .
statics	<code>schema.statics.findByEmail = function(email) {...}</code>	Adds a custom method callable on the model itself: <code>User.findByEmail(email)</code> .
virtuals	<code>schema.virtual("fullName").get(function() {...})</code>	Defines a computed field that isn't stored in the database, calculated on the fly (e.g. combining first + last name).
<code>index()</code>	<code>schema.index({ email: 1 })</code>	Creates a database index to speed up queries on that field.

8. Common Schema Field Options

Option	Example	What it does
<code>required</code>	<code>required: [true, 'email is required']</code>	Field must be present, or save/validation fails with the given message.
<code>unique</code>	<code>unique: true</code>	No two documents can share the same value for this field (enforced via a database index).
<code>sparse</code>	<code>sparse: true</code>	Used with <code>unique</code> on optional fields — allows multiple documents to have no value at all without conflicting.
<code>default</code>	<code>default: ""</code>	Value used automatically if none is provided.
<code>enum</code>	<code>enum: ["artist", "listener"]</code>	Restricts a string field to a fixed set of allowed values.
<code>select</code>	<code>select: false</code>	Field is excluded from query results by default (e.g. <code>password</code>) — must be explicitly requested with <code>.select("+password")</code> .
<code>ref</code>	<code>ref: "users"</code>	Marks an <code>ObjectId</code> field as a reference to another collection, enabling <code>.populate()</code> .

set	set: (val) => val.trim()	A function that transforms a value every time it's assigned, before validation/saving.
get	get: (val) => val.toFixed(2)	A function that transforms a value every time it's read back out.

Quick Decision Guide

Situation	Use
Creating a brand-new, independent document	.create()
Might already exist, and you want update-or-create in one step	.findOneAndUpdate() + upsert: true
Need pre("save") hooks or isModified() to run	.save() (fetch → modify → save)
Just need document(s), no modification	.find() / .findOne() / .findById()
Need to enforce validation rules on an update	Add runValidators: true
Need the updated (not original) document back	Add returnDocument: 'after'